CFM

Campus Formations et Métiers — Bobigny 93

Mini LMS Pédagogique

Documentation Technique

Plateforme de gestion de l'apprentissage développée avec Laravel 12, Bootstrap 5 et intégration IA via OpenRouter.

Gestion des formations, chapitres, quiz, apprenants et notes.

Version 1.0 — **Date** 09/04/2026

Framework Laravel 12 — **Base de données** SQLite

Interface Bootstrap 5.3 — **IA** OpenRouter / Claude 3 Haiku

Campus Formations et Métiers — Bobigny, Seine-Saint-Denis (93) — Documentation confidentielle © 09/04/2026

Documentation Technique — CFM LMS

Campus Formations et Métiers — Bobigny 93 Version 1.0 — Avril 2026

Table des matières

1. Vue d'ensemble du projet
2. Architecture applicative
3. Schéma de base de données
4. Migrations
5. Modèles Eloquent et relations
6. Contrôleurs
7. Routes
8. Seeders
9. Fonctionnalité Générateur IA
10. Installation et démarrage

1. Vue d'ensemble du projet

Description

Le **CFM LMS** (Learning Management System) est une plateforme pédagogique en ligne développée avec Laravel 12 pour le Campus Formations et Métiers de Bobigny (93). Elle permet à des administrateurs de créer et gérer des formations complètes (chapitres, contenus, quiz, notes), et à des apprenants de les consulter et d'y passer des évaluations.

Fonctionnalités principales

| Fonctionnalité | Rôle concerné | |---|---| | Création et gestion de formations | Admin | | Organisation en chapitres et sous-chapitres | Admin | | Création de quiz avec questions à choix multiple | Admin | | Inscription des apprenants aux formations | Admin | | Attribution et suivi des notes | Admin | | Génération automatique de formations par IA | Admin | | Consultation des formations et contenus | Apprenant | | Passage de quiz et visualisation des résultats | Apprenant | | Consultation de son relevé de notes | Apprenant |

Stack technique

| Composant | Technologie | |---|---| | Framework backend | Laravel 12 | | Authentification | Laravel Breeze (stack Blade) | | Base de données | SQLite (développement) | | Frontend | Bootstrap 5.3 via CDN | | Icônes | Bootstrap Icons 1.11 via CDN | |

Typographie | Inter (Google Fonts) | | API IA externe | OpenRouter (modèle `anthropic/claude-3-haiku`) | | HTTP client | Laravel Http Facade |

2. Architecture applicative

Structure des répertoires (fichiers clés)

```
mini-lms/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Admin/              # Contrôleurs espace administrateur
│   │   │   │   ├── AiGeneratorController.php
│   │   │   │   ├── ApprenantController.php
│   │   │   │   ├── ChapitreController.php
│   │   │   │   ├── ContenuController.php
│   │   │   │   ├── DashboardController.php
│   │   │   │   ├── FormationController.php
│   │   │   │   ├── NoteController.php
│   │   │   │   ├── QuestionController.php
│   │   │   │   └── QuizController.php
│   │   │   ├── Apprenant/          # Contrôleurs espace apprenant
│   │   │   │   ├── DashboardController.php
│   │   │   │   ├── FormationController.php
│   │   │   │   ├── NoteController.php
│   │   │   │   └── QuizController.php
│   │   │   └── Auth/               # Contrôleurs d'authentification (Breeze)
│   │   └── Middleware/
│   │       ├── AdminMiddleware.php
│   │       └── ApprenantMiddleware.php
│   ├── Models/                     # Modèles Eloquent
│   │   ├── User.php
│   │   ├── Formation.php
│   │   ├── Chapitre.php
│   │   ├── SousChapitre.php
│   │   ├── Contenu.php
│   │   ├── Quiz.php
│   │   ├── Question.php
│   │   ├── Reponse.php
│   │   ├── QuizResult.php
│   │   └── Note.php
│   └── View/Components/
│       ├── AppLayout.php
│       └── GuestLayout.php
├── bootstrap/
│   └── app.php                     # Enregistrement des middlewares
├── config/
│   └── services.php                # Clé API OpenRouter
├── database/
│   ├── migrations/                 # 13 migrations
│   └── seeders/
│       └── DatabaseSeeder.php
├── resources/views/
│   ├── layouts/
│   │   ├── app.blade.php           # Layout principal (navbar + sidebar)
│   │   └── guest.blade.php         # Layout login (split-screen)
```

```
│   ├── admin/                    # Vues espace admin
│   └── apprenant/                # Vues espace apprenant
└── routes/
    └── web.php
```

Système de rôles

L'application distingue deux rôles stockés dans la colonne `role` de la table `users` :

- **`admin`** : accès complet à la gestion des formations, apprenants, quiz, notes et au générateur IA.
- **`apprenant`** : accès en lecture aux formations auxquelles il est inscrit, passage des quiz, consultation de ses notes.

Le contrôle d'accès est assuré par deux middlewares enregistrés comme alias dans `bootstrap/app.php` :

```
$middleware->alias([
    'admin'     => \App\Http\Middleware\AdminMiddleware::class,
    'apprenant' => \App\Http\Middleware\ApprenantMiddleware::class,
]);
```

Chaque middleware vérifie `auth()->user()->isAdmin()` (respectivement `isApprenant()`) et retourne un `abort(403)` si la condition n'est pas satisfaite.

Flux d'authentification

1. L'utilisateur soumet le formulaire de login (`/login`).
2. `AuthenticatedSessionController@store` vérifie les identifiants.
3. Selon le rôle, la redirection se fait vers `admin.dashboard` ou `apprenant.dashboard`.

```
$route = Auth::user()->isAdmin() ? 'admin.dashboard' : 'apprenant.dashboard';
return redirect()->intended(route($route));
```

3. Schéma de base de données

Diagramme des relations

```
users ──────────────────────────────────────────────────────┐
  │                                                          │
  │ belongsToMany (formation_user)    hasMany                │ hasMany
  ▼                                   ▼                      ▼
formations ─── hasMany ──► chapitres      quiz_results        notes
  │                           │
  │ hasMany                   │ hasMany
  ▼                           ▼
notes                     sous_chapitres ─── hasOne ──► quizzes
                                                          │
                                                          │ hasMany
                                                          ▼
                                                        questions
                                                          │
                                                          │ hasMany
                                                          ▼
                                                        reponses
```

Tables et colonnes

`users`

Colonne	Type	Contraintes	Description
`id`	bigint	PK, auto-increment	Identifiant unique
`name`	varchar(255)	NOT NULL	Nom complet
`email`	varchar(255)	UNIQUE, NOT NULL	Adresse e-mail
`email_verified_at`	timestamp	NULL	Date de vérification e-mail
`password`	varchar(255)	NOT NULL	Mot de passe haché (bcrypt)
`role`	enum	NOT NULL, défaut: `apprenant`	`admin` ou `apprenant`
`remember_token`	varchar(100)	NULL	Token "se souvenir de moi"
`created_at` / `updated_at`	timestamp		Horodatages automatiques

`formations`

Colonne	Type	Contraintes	Description
`id`	bigint	PK	Identifiant unique
`nom`	varchar(255)	NOT NULL	Intitulé de la formation
`description`	text	NULL	Description détaillée
`niveau`	varchar(255)	défaut: `débutant`	`débutant`, `intermédiaire` ou `avancé`
`duree`	integer	NULL	Durée estimée en heures
`created_at` / `updated_at`	timestamp		

`formation_user` ***(table pivot)***

Colonne	Type	Contraintes	Description
`id`	bigint	PK	
`formation_id`	bigint	FK → formations, CASCADE	
`user_id`	bigint	FK → users, CASCADE	
`inscrit_le`	timestamp	défaut: NOW()	Date d'inscription
`created_at` / `updated_at`	timestamp		

`chapitres`

Colonne	Type	Contraintes	Description
`id`	bigint	PK	
`formation_id`	bigint	FK → formations, CASCADE	
`titre`	varchar(255)	NOT NULL	Titre du chapitre
`description`	text	NULL	Description optionnelle
`ordre`	integer	défaut: 0	Position dans la formation
`created_at` / `updated_at`	timestamp		

`sous_chapitres`

Colonne	Type	Contraintes	Description
`id`	bigint	PK	
`chapitre_id`	bigint	FK → chapitres, CASCADE	
`titre`	varchar(255)	NOT NULL	Titre du sous-chapitre
`contenu`	longtext	NULL	Contenu pédagogique HTML
`ordre`	integer	défaut: 0	Position dans le chapitre
`created_at` / `updated_at`	timestamp		

`contenus`

Colonne	Type	Contraintes	Description
`id`	bigint	PK	
`sous_chapitre_id`	bigint	FK → sous_chapitres, CASCADE	
`titre`	varchar(255)	NOT NULL	Titre du bloc de contenu
`texte`	longtext	NULL	Texte libre
`lien_ressource`	varchar(255)	NULL	URL vers une ressource externe
`importe_ia`	boolean	défaut: false	Marqueur de génération IA
`created_at` / `updated_at`	timestamp		

`quizzes`

Colonne	Type	Contraintes	Description
`id`	bigint	PK	
`sous_chapitre_id`	bigint	FK → sous_chapitres, CASCADE	Sous-chapitre rattaché
`titre`	varchar(255)	NOT NULL	Titre du quiz
`description`	text	NULL	Description
`created_at` / `updated_at`	timestamp		

`questions`

Colonne	Type	Contraintes	Description
`id`	bigint	PK	
`quiz_id`	bigint	FK → quizzes, CASCADE	
`question`	text	NOT NULL	Énoncé de la question
`ordre`	integer	défaut: 0	Position dans le quiz
`created_at` / `updated_at`	timestamp		

`reponses`

Colonne	Type	Contraintes	Description
`id`	bigint	PK	
`question_id`	bigint	FK → questions, CASCADE	
`texte`	varchar(255)	NOT NULL	Texte de la proposition
`est_correcte`	boolean	défaut: false	Indique la bonne réponse
`created_at` / `updated_at`	timestamp		

`quiz_results`

Colonne	Type	Contraintes	Description
`id`	bigint	PK	
`user_id`	bigint	FK → users, CASCADE	Apprenant
`quiz_id`	bigint	FK → quizzes, CASCADE	Quiz passé
`score`	integer	NOT NULL	Nombre de bonnes réponses
`total`	integer	NOT NULL	Total de questions
`created_at` / `updated_at`	timestamp		

`notes`

Colonne	Type	Contraintes	Description
`id`	bigint	PK	
`user_id`	bigint	FK → users, CASCADE	Apprenant évalué
`formation_id`	bigint	FK → formations, CASCADE	Formation concernée
`matiere`	varchar(255)	NOT NULL	Nom de la matière / module
`note`	decimal(4,2)	NOT NULL	Note sur 20
`commentaire`	text	NULL	Commentaire de l'évaluateur
`created_at` / `updated_at`	timestamp		

---

4. Migrations

Les migrations sont exécutées dans l'ordre alphabétique de leur nom de fichier.

Fichier	Table(s) créée(s)	Description
`0001_01_01_000000_create_users_table.php`	`users`, `password_reset_tokens`, `sessions`	Utilisateurs avec colonne `role` (enum), tokens de réinitialisation de mot de passe, sessions serveur
`0001_01_01_000001_create_cache_table.php`	`cache`, `cache_locks`	Cache applicatif Laravel (généré par Breeze)

`0001_01_01_000002_create_jobs_table.php` | `jobs`, `job_batches`, `failed_jobs` | File de jobs asynchrones Laravel (généré par Breeze) | | `2024_01_01_100000_create_formations_table.php` | `formations` | Formations pédagogiques avec niveau et durée | | `2024_01_01_100001_create_formation_user_table.php` | `formation_user` | Table pivot many-to-many formations ↔ apprenants avec `inscrit_le` | | `2024_01_01_100002_create_chapitres_table.php` | `chapitres` | Chapitres rattachés à une formation, ordonnés | | `2024_01_01_100003_create_sous_chapitres_table.php` | `sous_chapitres` | Sous-chapitres avec contenu HTML longtext | | `2024_01_01_100004_create_contenus_table.php` | `contenus` | Blocs de contenu additionnels avec marqueur `importe_ia` | | `2024_01_01_100005_create_quizzes_table.php` | `quizzes` | Quiz rattachés à un sous-chapitre | | `2024_01_01_100006_create_questions_table.php` | `questions` | Questions de quiz ordonnées | | `2024_01_01_100007_create_reponses_table.php` | `reponses` | Propositions de réponse avec booléen `est_correcte` | | `2024_01_01_100008_create_quiz_results_table.php` | `quiz_results` | Résultats enregistrés (score/total) par utilisateur et quiz | | `2024_01_01_100009_create_notes_table.php` | `notes` | Notes sur 20 attribuées par l'admin à un apprenant pour une formation |

Toutes les clés étrangères sont définies avec `onDelete('cascade')` : la suppression d'une entité parente supprime automatiquement toutes ses entités enfants.

---

5. Modèles Eloquent et relations

`User`

Table : `users` **Fillable** : `name`, `email`, `password`, `role` **Casts** : `email_verified_at` → datetime, `password` → hashed

Méthode	Type	Modèle lié	Description
`formations()`	`belongsToMany`	`Formation`	Via pivot `formation_user`, expose `inscrit_le`
`quizResults()`	`hasMany`	`QuizResult`	Tous les résultats de quiz de l'utilisateur
`notes()`	`hasMany`	`Note`	Toutes les notes reçues
`isAdmin()`	helper	—	Retourne `true` si `role === 'admin'`
`isApprenant()`	helper	—	Retourne `true` si `role === 'apprenant'`

`Formation`

Table : `formations` **Fillable** : `nom`, `description`, `niveau`, `duree`

Méthode	Type	Modèle lié	Description
`chapitres()`	`hasMany`	`Chapitre`	Ordonnés par `ordre` ASC
`apprenants()`	`belongsToMany`	`User`	Via pivot `formation_user`
`notes()`	`hasMany`	`Note`	Notes liées à cette formation

`Chapitre`

Table : `chapitres` **Fillable** : `formation_id`, `titre`, `description`, `ordre`

Méthode	Type	Modèle lié	Description
`formation()`	`belongsTo`	`Formation`	Formation parente
`sousChapitres()`	`hasMany`	`SousChapitre`	Ordonnés par `ordre` ASC

`SousChapitre`

Table : `sous_chapitres` *(explicitement définie)* **Fillable** : `chapitre_id`, `titre`, `contenu`, `ordre`

| Méthode | Type | Modèle lié | Description | |---|---|---|---| | `chapitre()` | `belongsTo` | `Chapitre` | Chapitre parent | | `contenus()` | `hasMany` | `Contenu` | Blocs de contenu additionnels | | `quiz()` | `hasOne` | `Quiz` | Quiz unique rattaché (relation 1-1) |

`Contenu`

Table : `contenus` **Fillable** : `sous_chapitre_id`, `titre`, `texte`, `lien_ressource`, `importe_ia`

| Méthode | Type | Modèle lié | Description | |---|---|---|---| | `sousChapitre()` | `belongsTo` | `SousChapitre` | Sous-chapitre parent |

`Quiz`

Table : `quizzes` *(explicitement définie)* **Fillable** : `sous_chapitre_id`, `titre`, `description`

| Méthode | Type | Modèle lié | Description | |---|---|---|---| | `sousChapitre()` | `belongsTo` | `SousChapitre` | Clé étrangère `sous_chapitre_id` | | `questions()` | `hasMany` | `Question` | Ordonnées par `ordre` ASC | | `results()` | `hasMany` | `QuizResult` | Historique des résultats |

`Question`

Table : `questions` **Fillable** : `quiz_id`, `question`, `ordre`

| Méthode | Type | Modèle lié | Description | |---|---|---|---| | `quiz()` | `belongsTo` | `Quiz` | Quiz parent | | `reponses()` | `hasMany` | `Reponse` | Propositions de réponse |

`Reponse`

Table : `reponses` **Fillable** : `question_id`, `texte`, `est_correcte`

| Méthode | Type | Modèle lié | Description | |---|---|---|---| | `question()` | `belongsTo` | `Question` | Question parente |

`QuizResult`

Table : `quiz_results` *(explicitement définie)* **Fillable** : `user_id`, `quiz_id`, `score`, `total`

| Méthode | Type | Modèle lié | Description | |---|---|---|---| | `user()` | `belongsTo` | `User` | Apprenant concerné | | `quiz()` | `belongsTo` | `Quiz` | Quiz passé | | `getPourcentageAttribute()` | accessor | — | Calcule `(score / total) * 100`, arrondi à 1 décimale |

`Note`

Table : `notes` **Fillable** : `user_id`, `formation_id`, `matiere`, `note`, `commentaire`

| Méthode | Type | Modèle lié | Description | |---|---|---|---| | `user()` | `belongsTo` | `User` | Apprenant évalué | | `formation()` | `belongsTo` | `Formation` | Formation concernée |

---

6. Contrôleurs

Espace Administrateur (`App\Http\Controllers\Admin`)

`DashboardController`

| Méthode | Route | Description | |---|---|---| | `index()` | GET `/admin/dashboard` | Affiche le tableau de bord avec compteurs (formations, apprenants, quiz) et les 10 derniers résultats de quiz |

`FormationController`

Gestion CRUD complète des formations + gestion des inscriptions.

| Méthode | Route | Description | |---|---|---| | `index()` | GET `/admin/formations` | Liste toutes les formations avec compteurs de chapitres et apprenants inscrits | | `create()` | GET `/admin/formations/create` | Formulaire de création | | `store(Request)` | POST `/admin/formations` | Valide et persiste une nouvelle formation | | `show(Formation)` | GET `/admin/formations/{id}` | Détail : chapitres, sous-chapitres, quiz, liste des apprenants inscrits | | `edit(Formation)` | GET `/admin/formations/{id}/edit` | Formulaire d'édition | | `update(Request, Formation)` | PUT `/admin/formations/{id}` | Met à jour une formation | | `destroy(Formation)` | DELETE `/admin/formations/{id}` | Supprime une formation (cascade) | | `inscrire(Request, Formation)` | POST `/admin/formations/{id}/inscrire` | Inscrit un apprenant (`syncWithoutDetaching`) | | `desinscrire(Formation, User)` | DELETE `/admin/formations/{id}/desinscrire/{user}` | Désinscrit un apprenant (`detach`) |

Validation : `nom` requis, `niveau` parmi `débutant|intermédiaire|avancé` , `duree` entier optionnel.

`ChapitreController`

CRUD des chapitres.

| Méthode | Route | Description | |---|---|---| | `index()` | GET `/admin/chapitres` | Liste tous les chapitres avec leur formation | | `create(Request)` | GET `/admin/chapitres/create` | Formulaire (pré-sélection de formation via `?formation_id=`) | | `store(Request)` | POST `/admin/chapitres` | Crée et redirige vers la formation parente | | `show(Chapitre)` | GET `/admin/chapitres/{id}` | Détail avec sous-chapitres et quiz | | `edit(Chapitre)` | GET `/admin/chapitres/{id}/edit` | Formulaire d'édition | | `update(Request, Chapitre)` | PUT `/admin/chapitres/{id}` | Met à jour | | `destroy(Chapitre)` | DELETE `/admin/chapitres/{id}` | Supprime et redirige vers la formation parente |

`SousChapitreController`

CRUD des sous-chapitres. Similaire à `ChapitreController` .

| Méthode | Route | Description | |---|---|---| | `index()` | GET `/admin/sous-chapitres` | Liste tous les sous-chapitres | | `create(Request)` | GET `/admin/sous-chapitres/create` | Formulaire avec sélection du chapitre | | `store(Request)` | POST `/admin/sous-chapitres` | Crée et redirige vers le chapitre parent | | `show(SousChapitre)` | GET `/admin/sous-chapitres/{id}` | Détail du sous-chapitre | | `edit(SousChapitre)` | GET `/admin/sous-chapitres/{id}/edit` | Édition | | `update(Request, SousChapitre)` | PUT `/admin/sous-chapitres/{id}` | Mise à jour | | `destroy(SousChapitre)` | DELETE `/admin/sous-chapitres/{id}` | Suppression |

`QuizController`

CRUD des quiz.

Méthode	Route	Description
`index()`	GET `/admin/quizzes`	Liste tous les quiz avec nombre de questions et chemin formation
`create(Request)`	GET `/admin/quizzes/create`	Formulaire (pré-sélection sous-chapitre via query string)
`store(Request)`	POST `/admin/quizzes`	Crée le quiz et redirige vers sa page (pour ajouter des questions)
`show(Quiz)`	GET `/admin/quizzes/{id}`	Détail avec toutes les questions et réponses
`edit(Quiz)`	GET `/admin/quizzes/{id}/edit`	Formulaire d'édition
`update(Request, Quiz)`	PUT `/admin/quizzes/{id}`	Mise à jour
`destroy(Quiz)`	DELETE `/admin/quizzes/{id}`	Suppression (cascade sur questions et réponses)

`QuestionController`

CRUD des questions + gestion des réponses associées.

Méthode	Route	Description
`index()`	GET `/admin/questions`	Liste toutes les questions
`create(Request)`	GET `/admin/questions/create`	Formulaire de création
`store(Request)`	POST `/admin/questions`	Crée la question
`show(Question)`	GET `/admin/questions/{id}`	Détail avec ses réponses ; formulaire d'ajout de réponse
`edit(Question)`	GET `/admin/questions/{id}/edit`	Édition
`update(Request, Question)`	PUT `/admin/questions/{id}`	Mise à jour
`destroy(Question)`	DELETE `/admin/questions/{id}`	Suppression
`storeReponse(Request, Question)`	POST `/admin/questions/{id}/reponses`	Ajoute une réponse ; garantit l'unicité de la bonne réponse (reset des autres à `false` avant insertion)
`destroyReponse(Reponse)`	DELETE `/admin/reponses/{id}`	Supprime une réponse

`NoteController`

CRUD des notes.

Méthode	Route	Description
`index()`	GET `/admin/notes`	Liste toutes les notes avec apprenant et formation
`create()`	GET `/admin/notes/create`	Formulaire (sélection apprenant + formation)
`store(Request)`	POST `/admin/notes`	Valide et enregistre (note entre 0 et 20)
`show(Note)`	GET `/admin/notes/{id}`	Détail d'une note
`edit(Note)`	GET `/admin/notes/{id}/edit`	Édition
`update(Request, Note)`	PUT `/admin/notes/{id}`	Mise à jour
`destroy(Note)`	DELETE `/admin/notes/{id}`	Suppression

`ApprenantController`

Consultation des apprenants (lecture seule).

Méthode	Route	Description
`index()`	GET `/admin/apprenants`	Liste tous les apprenants avec compteur de formations
`show(User)`	GET `/admin/apprenants/{id}`	Profil complet : formations, notes, résultats de quiz

---

`AiGeneratorController`

Voir section 9 pour le détail complet.

Méthode	Route	Description
`index()`	GET `/admin/ai-generator`	Affiche le formulaire de génération
`generate(Request)`	POST `/admin/ai-generator`	Lance la génération IA et persiste en base

Espace Apprenant (`App\Http\Controllers\Apprenant`)

`DashboardController`

| Méthode | Route | Description | |---|---|---| | `index()` | GET `/apprenant/dashboard` | Tableau de bord : formations inscrites, 5 dernières notes, 5 derniers résultats de quiz |

`FormationController`

Consultation des formations (accès restreint aux formations inscrites).

| Méthode | Route | Description | |---|---|---| | `index()` | GET `/apprenant/formations` | Liste les formations auxquelles l'apprenant est inscrit | | `show(Formation)` | GET `/apprenant/formations/{id}` | Détail : arborescence chapitres / sous-chapitres / quiz. Vérifie l'inscription via `formations()->where(...)->exists()` | | `sousChapitre(SousChapitre)` | GET `/apprenant/sous-chapitres/{id}` | Affiche le contenu d'un sous-chapitre. Vérifie l'inscription à la formation parente |

`QuizController`

Passage des quiz et consultation des résultats.

| Méthode | Route | Description | |---|---|---| | `show(Quiz)` | GET `/apprenant/quiz/{id}` | Affiche le quiz avec ses questions. Vérifie l'inscription. Signale si déjà passé | | `soumettre(Request, Quiz)` | POST `/apprenant/quiz/{id}/soumettre` | Calcule le score en comparant les réponses soumises aux `est_correcte`, persiste `QuizResult`, redirige vers le résultat | | `resultat(Quiz, QuizResult)` | GET `/apprenant/quiz/{id}/resultat/{result}` | Affiche le résultat détaillé (score, pourcentage, correction) |

`NoteController`

| Méthode | Route | Description | |---|---|---| | `index()` | GET `/apprenant/notes` | Liste toutes les notes de l'apprenant connecté |

---

7. Routes

Groupe Admin

Middleware : `auth`, `admin` — **Préfixe** : `/admin` — **Nom** : `admin.`

```
GET    /admin/dashboard                              → admin.dashboard
GET    /admin/ai-generator                           → admin.ai-generator
POST   /admin/ai-generator                           → admin.ai-generator.generate

GET    /admin/formations                             → admin.formations.index
GET    /admin/formations/create                      → admin.formations.create
POST   /admin/formations                             → admin.formations.store
GET    /admin/formations/{formation}                 → admin.formations.show
GET    /admin/formations/{formation}/edit            → admin.formations.edit
PUT    /admin/formations/{formation}                 → admin.formations.update
DELETE /admin/formations/{formation}                 → admin.formations.destroy
POST   /admin/formations/{formation}/inscrire        → admin.formations.inscrire
DELETE /admin/formations/{formation}/desinscrire/{user} → admin.formations.desinscrire

GET    /admin/chapitres                              → admin.chapitres.index
GET    /admin/chapitres/create                       → admin.chapitres.create
POST   /admin/chapitres                              → admin.chapitres.store
GET    /admin/chapitres/{chapitre}                   → admin.chapitres.show
GET    /admin/chapitres/{chapitre}/edit              → admin.chapitres.edit
PUT    /admin/chapitres/{chapitre}                   → admin.chapitres.update
DELETE /admin/chapitres/{chapitre}                   → admin.chapitres.destroy

GET    /admin/sous-chapitres                         → admin.sous-chapitres.index
GET    /admin/sous-chapitres/create                  → admin.sous-chapitres.create
POST   /admin/sous-chapitres                         → admin.sous-chapitres.store
GET    /admin/sous-chapitres/{sous-chapitre}         → admin.sous-chapitres.show
GET    /admin/sous-chapitres/{sous-chapitre}/edit    → admin.sous-chapitres.edit
PUT    /admin/sous-chapitres/{sous-chapitre}         → admin.sous-chapitres.update
DELETE /admin/sous-chapitres/{sous-chapitre}         → admin.sous-chapitres.destroy

GET    /admin/contenus                               → admin.contenus.index
GET    /admin/contenus/create                        → admin.contenus.create
POST   /admin/contenus                               → admin.contenus.store
GET    /admin/contenus/{contenu}                     → admin.contenus.show
GET    /admin/contenus/{contenu}/edit                → admin.contenus.edit
PUT    /admin/contenus/{contenu}                     → admin.contenus.update
DELETE /admin/contenus/{contenu}                     → admin.contenus.destroy

GET    /admin/quizzes                                → admin.quizzes.index
GET    /admin/quizzes/create                         → admin.quizzes.create
POST   /admin/quizzes                                → admin.quizzes.store
GET    /admin/quizzes/{quiz}                         → admin.quizzes.show
GET    /admin/quizzes/{quiz}/edit                    → admin.quizzes.edit
PUT    /admin/quizzes/{quiz}                         → admin.quizzes.update
DELETE /admin/quizzes/{quiz}                         → admin.quizzes.destroy

GET    /admin/questions                              → admin.questions.index
GET    /admin/questions/create                       → admin.questions.create
POST   /admin/questions                              → admin.questions.store
GET    /admin/questions/{question}                   → admin.questions.show
GET    /admin/questions/{question}/edit              → admin.questions.edit
PUT    /admin/questions/{question}                   → admin.questions.update
DELETE /admin/questions/{question}                   → admin.questions.destroy
POST   /admin/questions/{question}/reponses          → admin.questions.reponses.store
```

```
DELETE /admin/reponses/{reponse}                    → admin.reponses.destroy

GET    /admin/apprenants                            → admin.apprenants.index
GET    /admin/apprenants/{user}                     → admin.apprenants.show

GET    /admin/notes                                 → admin.notes.index
GET    /admin/notes/create                          → admin.notes.create
POST   /admin/notes                                 → admin.notes.store
GET    /admin/notes/{note}                          → admin.notes.show
GET    /admin/notes/{note}/edit                     → admin.notes.edit
PUT    /admin/notes/{note}                          → admin.notes.update
DELETE /admin/notes/{note}                          → admin.notes.destroy
```

Groupe Apprenant

Middleware : `auth`, `apprenant` — **Préfixe** : `/apprenant` — **Nom** : `apprenant.`

```
GET  /apprenant/dashboard                           → apprenant.dashboard
GET  /apprenant/formations                          → apprenant.formations.index
GET  /apprenant/formations/{formation}              → apprenant.formations.show
GET  /apprenant/sous-chapitres/{sousChapitre}       → apprenant.sous-chapitres.show
GET  /apprenant/quiz/{quiz}                         → apprenant.quiz.show
POST /apprenant/quiz/{quiz}/soumettre               → apprenant.quiz.soumettre
GET  /apprenant/quiz/{quiz}/resultat/{result}       → apprenant.quiz.resultat
GET  /apprenant/notes                               → apprenant.notes.index
```

Routes d'authentification (Breeze)

```
GET  /login              → login
POST /login              → login (traitement)
POST /logout             → logout
GET  /forgot-password    → password.request
POST /forgot-password    → password.email
GET  /reset-password/{token} → password.reset
POST /reset-password     → password.store
GET  /profile            → profile.edit
PATCH /profile           → profile.update
DELETE /profile          → profile.destroy
```

Route racine

```
GET /   → Redirige vers admin.dashboard ou apprenant.dashboard selon le rôle
          (ou vers /login si non authentifié)
```

---

8. Seeders

`DatabaseSeeder`

Fichier : `database/seeders/DatabaseSeeder.php`

Le seeder crée un jeu de données de démonstration complet prêt à l'emploi.

Utilisateurs créés

| Nom | E-mail | Mot de passe | Rôle | |---|---|---|---| | Admin LMS | admin@lms.fr | password | admin | | Alice Martin | alice@lms.fr | password | apprenant |

Formation de démonstration

Anglais — Maîtriser les verbes irréguliers

- Niveau : débutant
- Durée : 4 heures
- Alice Martin est automatiquement inscrite via `attach()`

Structure pédagogique créée

```
Formation : Anglais — Maîtriser les verbes irréguliers
├── Chapitre 1 : Introduction aux verbes irréguliers (ordre: 1)
│   └── Sous-chapitre 1.1 : Qu'est-ce qu'un verbe irrégulier ? (ordre: 1)
│       └── Contenu : Ressource — Liste complète (importe_ia: true)
└── Chapitre 2 : Les 10 verbes irréguliers indispensables (ordre: 2)
    ├── Sous-chapitre 2.1 : Tableau des 10 verbes (ordre: 1)
    │   ├── Contenu : Les 10 verbes — généré par IA (importe_ia: true)
    │   └── Quiz : Quiz — Les 10 verbes irréguliers indispensables
    │       └── 7 questions à 4 réponses chacune
    └── Sous-chapitre 2.2 : Méthodes de mémorisation (ordre: 2)
```

Quiz de démonstration

7 questions portant sur les prétérits et participes passés des verbes go, see, have, do, come, know, take. Chaque question comporte 4 propositions dont une seule correcte.

Note de démonstration

- Apprenant : Alice Martin
- Formation : Anglais — verbes irréguliers
- Matière : Verbes irréguliers anglais
- Note : 15.50/20
- Commentaire : "Très bonne participation, quelques erreurs sur les participes passés."

---

9. Fonctionnalité Générateur IA

Vue d'ensemble

Le générateur IA permet à un administrateur de créer automatiquement une formation complète (chapitres, sous-chapitres avec contenu HTML, quiz et questions) en décrivant simplement un sujet pédagogique. La génération repose sur l'API **OpenRouter** avec le modèle `anthropic/claude-3-haiku`.

Accès

- URL : `/admin/ai-generator`
- Contrôleur : `App\Http\Controllers\Admin\AiGeneratorController`
- Lien dans la barre latérale sous "Outils IA"

Paramètres du formulaire

| Champ | Type | Contraintes | Description | |---|---|---|---| | `sujet` | textarea | requis, 5–300 caractères | Description du sujet pédagogique | | `nb_chapitres` | number | 1–10 | Nombre de chapitres à générer | | `nb_sous_chapitres` | number | 1–5 | Nombre de sous-chapitres par chapitre | | `nb_questions` | number | 3–15 | Nombre de questions par quiz | | `quiz_par_chapitre` | checkbox | optionnel | Si coché : un quiz par chapitre ; sinon : un seul quiz global |

Fonctionnement technique

1. Construction du prompt

Le contrôleur génère dynamiquement un exemple de structure JSON conforme aux paramètres saisis, puis l'injecte dans un prompt d'ingénierie pédagogique :

```
Tu es un expert en ingénierie pédagogique. Génère un contenu de formation COMPLET
et DÉTAILLÉ sur le sujet : "{sujet}".
Réponds UNIQUEMENT avec un objet JSON valide.
[structure JSON exemple générée dynamiquement]
```

Le prompt impose :

- Contenu HTML pour chaque sous-chapitre (`h5` , `p` , `ul/li` , `table` , `pre/code`)
- Exactement 4 réponses par question, dont une seule correcte (valeur identique à `correcte`)
- Niveau parmi `débutant | intermédiaire | avancé`
- JSON strict sans markdown ni backticks

2. Appel API

```
Http::timeout(150)
    ->withoutVerifying()          // SSL désactivé (développement local)
    ->withHeaders([
        'Authorization' => 'Bearer ' . config('services.openrouter.key'),
        'Content-Type'  => 'application/json',
    ])
    ->post('https://openrouter.ai/api/v1/chat/completions', [
        'model'      => 'anthropic/claude-3-haiku',
        'max_tokens' => 6000,
        'messages'   => [['role' => 'user', 'content' => $prompt]],
    ]);
```

La clé API est lue depuis `config('services.openrouter.key')` → variable d'environnement `OPENROUTER_API_KEY` .

3. Nettoyage et validation du JSON

```
$cleanJson = trim(preg_replace('/^```(?:json)?\s*|\s*```$/m', '', $rawContent));
$data = json_decode($cleanJson, true);
```

Si le JSON est invalide ou si les clés attendues sont absentes, le contrôleur retourne l'utilisateur au formulaire avec un message d'erreur et la réponse brute de l'IA visible dans un bloc `<details>`.

4. Persistance en base de données

La méthode privée `createQuiz(array $qzData, int $sousChapitreId)` factorise la création d'un quiz avec ses questions et réponses. Le flux de persistance est le suivant :

```
1. Formation::create()
2. Pour chaque chapitre :
   a. Chapitre::create()
   b. Pour chaque sous-chapitre :
      - SousChapitre::create()
      - Si quiz_par_chapitre : createQuiz() rattaché au sous-chapitre cible
3. Si quiz global : createQuiz() rattaché au sous-chapitre désigné par sous_chapitre_index
```

5. Redirection

En cas de succès, l'admin est redirigé vers la page de la formation créée (`admin.formations.show`) avec un message flash de confirmation.

Configuration requise

Ajouter dans le fichier `.env` :

```
OPENROUTER_API_KEY=sk-or-v1-xxxxxxxxxxxxxxxxxxxxxxxx
```

Limites et considérations

- La génération peut durer **20 à 150 secondes** selon la taille du contenu. `set_time_limit(180)` est appelé en début de méthode.
- Le modèle `claude-3-haiku` est optimisé pour la rapidité. Pour un contenu plus riche, envisager `claude-3-sonnet` ou `claude-3-opus` (coût plus élevé, tokens plus nombreux).
- `max_tokens: 6000` peut être insuffisant pour de grandes structures (10 chapitres × 5 sous-chapitres). Adapter selon les besoins.
- L'option `withoutVerifying()` désactive la vérification du certificat SSL — à ne pas utiliser en production.

10. Installation et démarrage

Prérequis

- PHP >= 8.2
- Composer
- Node.js (optionnel — non utilisé, pas de build Vite)
- SQLite (inclus dans PHP)

Étapes d'installation

1. Cloner le projet

```
git clone <url-du-dépôt> mini-lms
cd mini-lms
```

2. Installer les dépendances PHP

```
composer install
```

3. Configurer l'environnement

```
cp .env.example .env
php artisan key:generate
```

Éditer `.env` et configurer a minima :

```
APP_NAME="CFM LMS"
APP_URL=http://localhost:8000

DB_CONNECTION=sqlite
# DB_DATABASE=database/database.sqlite   ← créé automatiquement

OPENROUTER_API_KEY=sk-or-v1-votre-cle-ici
```

4. Créer la base de données SQLite

```
touch database/database.sqlite
```

5. Exécuter les migrations

```
php artisan migrate
```

6. Charger les données de démonstration

```
php artisan db:seed
```

Cela crée les deux comptes de test :

- `admin@lms.fr` / `password` (administrateur)
- `alice@lms.fr` / `password` (apprenant)

7. Lancer le serveur de développement

```
php artisan serve
```

L'application est accessible sur **http://localhost:8000**.

Réinitialiser la base de données

Pour repartir d'une base vierge avec les données de démo :

```
php artisan migrate:fresh --seed
```

Arborescence des comptes de test

| Compte | URL après connexion | Accès | |---|---|---| | admin@lms.fr | `/admin/dashboard` | Gestion complète + générateur IA | | alice@lms.fr | `/apprenant/dashboard` | Formations inscrites, quiz, notes |

Résolution des problèmes courants

| Problème | Solution | |---|---| | `Route [dashboard] not defined` | Vider le cache : `php artisan route:clear` | | Erreur SSL lors de l'appel OpenRouter | `withoutVerifying()` déjà présent ; vérifier que la clé API est correcte dans `.env` | | Timeout sur la génération IA | `set_time_limit(180)` est configuré ; augmenter `max_execution_time` dans `php.ini` si nécessaire | | JSON invalide retourné par l'IA | Réessayer ; réduire `nb_chapitres` ou `nb_sous_chapitres` pour diminuer la taille de la réponse | | Table `formation_user` sans colonne `created_at` | Exécuter `php artisan migrate:fresh` pour recréer toutes les tables |

Documentation générée le 09 avril 2026 — CFM Campus Formations et Métiers, Bobigny 93.